



---

# **I3C CONTROLLER VERIFICATION**

---



MAY 7, 2026  
MIRAFRA TECHNOLOGY

# **Content**

|                  |                             |
|------------------|-----------------------------|
| <b>Chapter 1</b> | <b>Introduction</b>         |
| <b>Chapter 2</b> | <b>Architecture</b>         |
| <b>Chapter 3</b> | <b>Testbench Components</b> |
| <b>Chapter 4</b> | <b>Design Bugs</b>          |
| <b>Chapter 5</b> | <b>Coverage</b>             |
| <b>Chapter 6</b> | <b>Output</b>               |
| <b>Chapter 7</b> | <b>Conclusion</b>           |

# CHAPTER – 1

## **1. Introduction**

- The I3C Controller is a high-speed serial communication controller that acts as an Active Controller (Master) on the I3C bus.
- It supports SDR (Single Data Rate) transfers, Dynamic Address Assignment (DAA), and Common Command Codes (CCC).
- The controller communicates with multiple targets using open-drain and push-pull signaling modes.
- It ensures efficient data transfer, arbitration, and device management on the I3C bus.

### **1.1. Functionality**

| <b>Module</b>     | <b>Function</b>                                                               |
|-------------------|-------------------------------------------------------------------------------|
| i3c_Rx_FIFO       | Implements the receive FIFO for storing incoming I3C data                     |
| i3c_Tx_FIFO       | Implements the transmit FIFO for buffering outgoing I3C data.                 |
| i3c_bit_engine    | Handles low-level bit transfer operations and SDA/SCL signal control.         |
| i3c_cmd_ctrl      | Controls command decoding and execution flow for I3C operations.              |
| i3c_daa_fsm       | Implements the FSM responsible for Dynamic Address Assignment (DAA).          |
| i3c_reg_interface | Implements the register interface for APB-based DUT configuration and access. |
| i3c_scl_gen       | Generates the SCL clock required for I3C communication.                       |
| i3c_sdr_fsm       | Implements the FSM controlling SDR read and write operations.                 |

### **1.2. Operating Sequence**

- Bus Initialization: Controller checks bus idle condition. Both SCL and SDA must be high before starting any transaction
- START Generation: Controller generates START condition by pulling SDA low while SCL is high, indicating beginning of transfer.
- Address Phase: Controller transmits 7-bit target address along with R/W bit to indicate read or write operation.

- ACK/NACK Handling: Target responds on the 9th clock cycle. ACK (SDA low) allows transfer to continue, NACK terminates the transaction

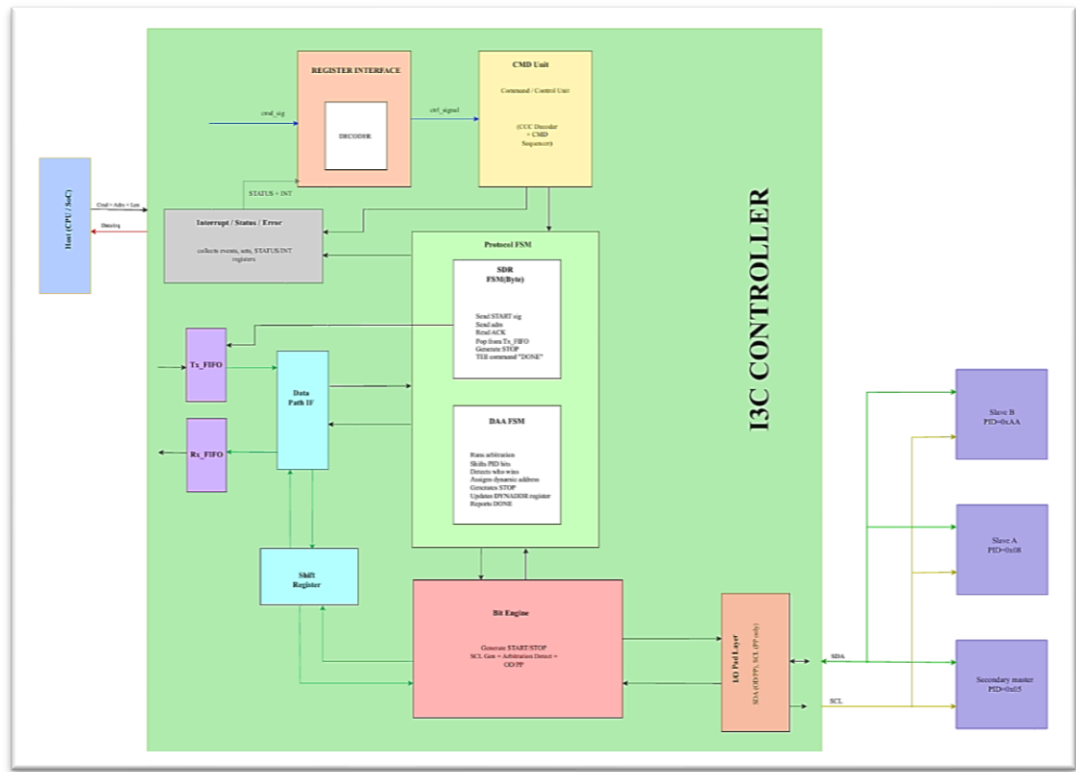
### 1.3 Working

- The project verifies an I3C Controller using a UVM-based testbench.
- The DUT acts as an Active Controller (Master) and controls communication on the I3C bus.
- The controller generates the SCL clock and manages bidirectional data transfer on SDA.
- The bus operates in two modes: open-drain for control phases and push-pull for data transfer.
- Before starting any transaction, the controller checks that SCL and SDA are both high (bus idle condition).
- The controller generates a START condition by pulling SDA low while SCL is high.
- The controller transmits a 7-bit target address along with a Read/Write bit.
- The target responds with ACK (SDA low) or NACK (SDA high) on the 9th clock cycle.
- If ACK is received, the transaction continues; if NACK is received, the controller terminates the transfer.
- During SDR write operation, the controller sends data bytes sequentially from the TX FIFO.
- Each transmitted byte is followed by an ACK from the target.
- If any byte is NACKed, the controller stops further transmission and generates STOP.
- During SDR read operation, the controller sends a read request and the target drives data on SDA.
- The controller samples incoming data on the rising edge of SCL.
- The controller sends ACK after each received byte except the last byte.
- For the last byte, the controller sends NACK and terminates the read operation.
- The controller generates a STOP condition by releasing SDA high while SCL is high.
- The controller uses TX FIFO to buffer outgoing data and RX FIFO to store received data.
- FIFO allows continuous data transfer without stalling the bus.
- The controller supports repeated START condition without releasing the bus.
- The controller supports Dynamic Address Assignment (DAA) using the ENTDA command.

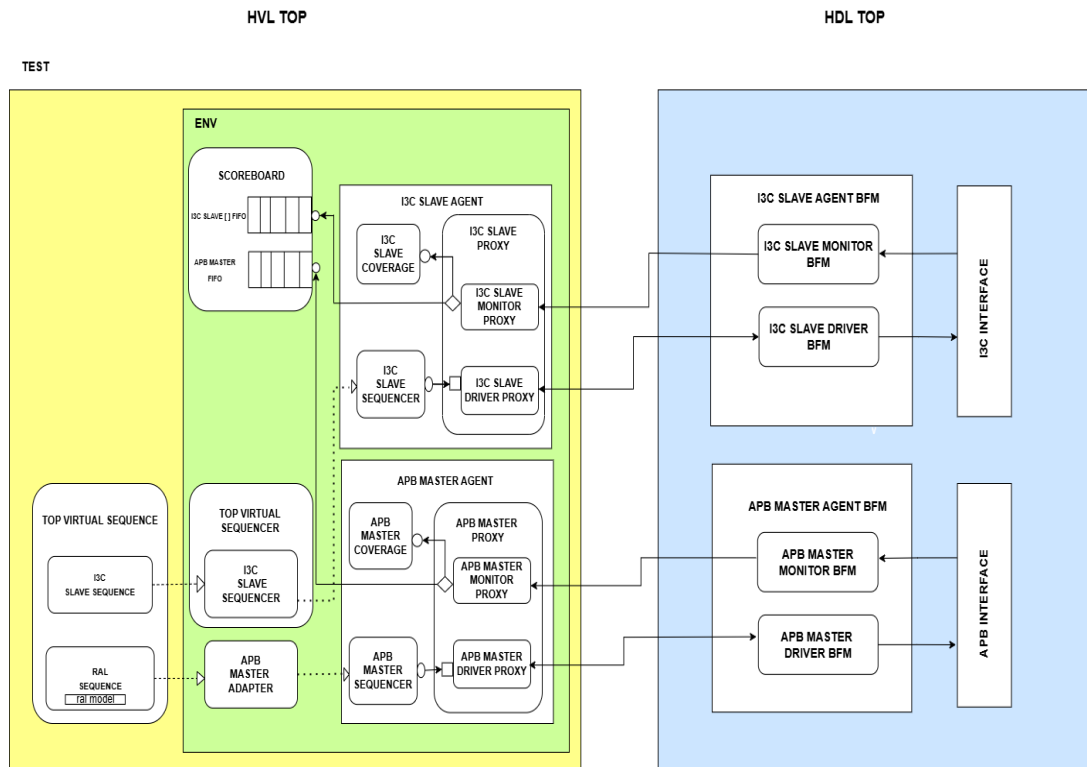
- The controller sends broadcast address (7'h7E + Write) to initiate DAA.
- The controller sends ENTDAACOMMAND (CCC = 0x07) to all targets.
- The controller generates a repeated START and sends (7'h7E + Read).
- All targets respond simultaneously with PID (48-bit), BCR (8-bit), and DCR (8-bit).
- Arbitration occurs on SDA using open-drain where logic '0' dominates logic '1'.
- The target with the lowest value wins arbitration and is selected by the controller.
- The controller assigns a 7-bit dynamic address along with a parity bit.
- The selected target acknowledges the assigned dynamic address.
- The controller stores PID, BCR, DCR, and dynamic address for future communication.
- The DAA process repeats until no more targets respond.
- The controller generates STOP condition after completing DAA.
- The controller detects error conditions such as NACK, arbitration loss, and FIFO overflow.
- The controller updates status signals and flags based on transaction outcome.
- The testbench verifies correct SDR read/write operations and DAA functionality.
- The testbench compares transmitted and received data to ensure data integrity.
- If transmitted and received data match, the transaction is considered successful.
- The verification environment monitors protocol compliance, timing, and signal behavior.

# CHAPTER – 2

## 1.3. Design Architecture



## 1.4. Testbench Architecture



## 2.3 Interface Signals

| Pin Name   | Direction | Width | Function                                                                               |
|------------|-----------|-------|----------------------------------------------------------------------------------------|
| wr_en      | output    | 1     | Write enable signal asserted by DUT to indicate a write operation to target memory     |
| rd_en      | output    | 1     | Read enable signal asserted by DUT to request data from target memory                  |
| addrs      | output    | 7     | Target address used for read/write transactions (dynamic address after DAA)            |
| w_reg_data | output    | 32    | Register write data from DUT (used for control/config registers via APB/CSR interface) |
| w_data     | output    | 8     | Write data sent from controller to target during SDR write operation                   |
| rd_data    | input     | 32    | Register read data provided to DUT (response for register read operations)             |
| r_data     | input     | 8     | Read data provided by target to DUT during SDR read operation                          |

## CHAPTER – 3

### Testbench Components

- `i3c_target_interface.sv`

Defines the connection between the DUT and the verification environment for the I3C protocol. It contains all the required I3C signals such as `scl_i`, `sda_i`, `scl_o`, and `sda_o` along with enable signals. It supports bidirectional communication required for I3C operation and acts as a bridge between the HDL domain and HVL domain. This interface is used by both driver and monitor BFM to drive and sample signals.

- `i3c_target_tx.sv`

Defines the transaction class representing a complete I3C transfer operation. It includes fields such as `targetAddress`, `operation` (READ/WRITE), `writeData`, `readData`, and `acknowledgment status`. It supports both Dynamic Address Assignment (DAA) and SDR operations. This transaction object is used for communication between sequencer, driver, monitor, and scoreboard.

- `i3c_target_sequence.sv`

Responsible for generating I3C transactions to stimulate the DUT. It creates different types of transactions such as DAA, SDR write, and SDR read. The sequence randomizes data and control fields to improve coverage and controls the order and timing of transactions.

- `i3c_target_sequencer.sv`

Acts as a bridge between the sequence and the driver. It receives transaction items from the sequence and forwards them to the driver. It maintains synchronization and ensures proper ordering of transactions during simulation.

- `i3c_target_driver_proxy.sv`

Receives transaction objects from the sequencer and converts them into struct format required by the BFM. It communicates with the driver BFM using a virtual interface and ensures that high-level transactions are correctly translated into low-level signal operations.

- `i3c_target_driver_bfm.sv`

Drives the actual I3C signals (SCL and SDA) to the DUT. It implements protocol behavior such as START and STOP conditions, address phase, DAA handling using broadcast address (0x7E), and SDR read/write operations. It works at the signal level and directly interfaces with DUT pins.



- `i3c_target_monitor_proxy.sv`

Receives sampled data from the monitor BFM and converts it into transaction objects. It sends these transactions to the scoreboard and coverage components using analysis ports. It acts as a bridge between signal-level monitoring and high-level verification.

- `i3c_target_monitor_bfm.sv`

Monitors I3C bus signals and captures SDA and SCL transitions. It extracts address, data, and acknowledgment information from the bus activity and sends the sampled data to the monitor proxy.

- `i3c_target_agent.sv`

Combines sequencer, driver proxy, monitor proxy, and coverage into a single component. It models the I3C slave behavior and operates in active mode to generate and apply stimulus to the DUT.

- `apb_master_interface.sv`

Defines APB interface signals such as PADDR, PWDATA, PRDATA, PSEL, and PENABLE. It is used for accessing DUT configuration registers and provides communication between APB agent and DUT.

- `apb_master_seq_item.sv`

Defines the APB transaction class including address, write data, read data, and control fields. It is used for performing register read and write operations.

- `apb_master_sequence.sv`

Generates APB read and write transactions. It is used to configure DUT registers such as command, address, and control fields before I3C operations begin.

- `apb_master_sequencer.sv`

Sends APB transactions from the sequence to the driver and ensures proper synchronization of register operations.

- `apb_master_driver_proxy.sv`

Converts APB transaction objects into signal-level operations and communicates with the APB driver BFM.

- `apb_master_driver_bfm.sv`

Drives APB signals to the DUT and performs register read/write operations according to the APB protocol.

- `apb_master_monitor_bfm.sv`

Monitors APB transactions, captures register activity, and sends observed data to the monitor proxy.

- `apb_master_agent.sv`

Integrates APB sequencer, driver, and monitor. It is responsible for configuring the DUT and operates in active mode.

- `i3c_scoreboard.sv`

Performs functional verification by comparing expected and actual results. It verifies DAA and SDR operations and ensures correctness of read/write data transactions.

- `i3c_target_coverage.sv`

Collects functional coverage including operation type, address range, data size, and acknowledgment conditions. It ensures all protocol scenarios are exercised.

- `i3c_virtual_sequencer.sv`

Coordinates multiple agents such as I3C and APB. It allows synchronized execution of transactions and acts as a central control unit.

- `i3c_daa_sdr_virtual_seq.sv`

Implements system-level verification flow including Dynamic Address Assignment followed by SDR write and read operations. It validates end-to-end functionality.

- `i3c_env.sv`

Instantiates all agents, scoreboard, and coverage components. It connects analysis ports and defines the overall structure of the verification environment.

- `i3c_base_test.sv`

Serves as the base test class. It creates and configures the environment and provides a reusable framework for different test cases.

- `i3c_daa_sdr_test.sv`

Top-level test case used for validating DAA and SDR operations. It starts the virtual sequence and controls simulation execution using UVM phases.

- `i3c_env_pkg.sv`

Central package file that includes all environment components such as agents, sequences, scoreboard, and configuration classes. It simplifies compilation and reuse.

- `i3c_globals_pkg.sv`

Contains global constants, parameters, and enumerations such as READ/WRITE and ACK/NACK. It ensures consistency across the entire testbench.

## CHAPTER – 4

### Bug Found

#### 4.1 Register Readback Behaviour – BUG

- According to the expected design functionality, all read-write (RW) registers in the DUT should support both write and read access, allowing verification through readback operations.
- Upon analyzing the DUT registers, it was observed that the readback logic has not been implemented for certain registers, specifically the CTRL and WDATA\_B registers.
- Due to the absence of readback support, the values written into these registers cannot be verified through read operations, leading to incomplete functionality and limiting effective validation of register behavior.
- This issue has been identified and reported, indicating that read access for these RW registers needs to be properly implemented in the DUT to ensure correct functionality and verification completeness.

#### 4.2 Register Parameter Inclusion Issue – BUG

- During compilation of the DUT and verification environment, compilation errors were observed related to register instance references used in the design code.
- Upon analysis, it was identified that certain register instances, specifically the CTRL register, were not included or declared in the parameter/configuration file used for register definitions.
- Since the design code internally uses the CTRL register instance, the absence of its declaration in the parameter file resulted in unresolved reference and compilation failures during build execution.
- This issue was identified as a design integration bug, indicating that all register instances used in the RTL design must be properly declared and mapped in the parameter/register definition files to ensure successful compilation and correct functionality.

## CHAPTER – 5

### DUT coverage

#### Local Instance Coverage Details:

| Total Coverage:                 |      |      |        |        | 69.26%  | 91.97%   |
|---------------------------------|------|------|--------|--------|---------|----------|
| Coverage Type                   | Bins | Hits | Misses | Weight | % Hit   | Coverage |
| <a href="#">Statements</a>      | 8    | 8    | 0      | 1      | 100.00% | 100.00%  |
| <a href="#">Branches</a>        | 12   | 12   | 0      | 1      | 100.00% | 100.00%  |
| <a href="#">FEC Expressions</a> | 7    | 7    | 0      | 1      | 100.00% | 100.00%  |
| <a href="#">Toggles</a>         | 614  | 417  | 197    | 1      | 67.91%  | 67.91%   |

#### Recursive Hierarchical Coverage Details:

| Total Coverage: |      |      |        |        | 60.79%  | 82.24%   |
|-----------------|------|------|--------|--------|---------|----------|
| Coverage Type   | Bins | Hits | Misses | Weight | % Hit   | Coverage |
| Statements      | 379  | 370  | 9      | 1      | 97.62%  | 97.62%   |
| Branches        | 289  | 269  | 20     | 1      | 93.07%  | 93.07%   |
| FEC Expressions | 32   | 29   | 3      | 1      | 90.62%  | 90.62%   |
| FEC Conditions  | 46   | 39   | 7      | 1      | 84.78%  | 84.78%   |
| Toggles         | 2220 | 1088 | 1132   | 1      | 49.00%  | 49.00%   |
| FSMs            | 87   | 61   | 26     | 1      | 70.11%  | 70.11%   |
| States          | 27   | 27   | 0      | 1      | 100.00% | 100.00%  |
| Transitions     | 60   | 34   | 26     | 1      | 56.66%  | 56.66%   |

#### Local Instance Coverage Details:

| Total Coverage:                 |      |      |        |        | 68.95%  | 91.89%   |
|---------------------------------|------|------|--------|--------|---------|----------|
| Coverage Type                   | Bins | Hits | Misses | Weight | % Hit   | Coverage |
| <a href="#">Statements</a>      | 8    | 8    | 0      | 1      | 100.00% | 100.00%  |
| <a href="#">Branches</a>        | 12   | 12   | 0      | 1      | 100.00% | 100.00%  |
| <a href="#">FEC Expressions</a> | 7    | 7    | 0      | 1      | 100.00% | 100.00%  |
| <a href="#">Toggles</a>         | 614  | 415  | 199    | 1      | 67.58%  | 67.58%   |

## Functional Coverage

### Coverage Summary by Type:

| Total Coverage:             |      |      |        |        | 45.03%  | <b>65.25%</b>  |
|-----------------------------|------|------|--------|--------|---------|----------------|
| Coverage Type               | Bins | Hits | Misses | Weight | % Hit   | Coverage       |
| <a href="#">Covergroups</a> | 34   | 34   | 0      | 1      | 100.00% | <b>100.00%</b> |
| Statements                  | 3594 | 1667 | 1927   | 1      | 46.38%  | <b>46.38%</b>  |
| Branches                    | 2078 | 624  | 1454   | 1      | 30.02%  | <b>30.02%</b>  |
| FEC Expressions             | 34   | 29   | 5      | 1      | 85.29%  | <b>85.29%</b>  |
| FEC Conditions              | 96   | 62   | 34     | 1      | 64.58%  | <b>64.58%</b>  |
| Toggles                     | 2668 | 1392 | 1276   | 1      | 52.17%  | <b>52.17%</b>  |
| FSMs                        | 87   | 61   | 26     | 1      | 70.11%  | <b>78.33%</b>  |
| States                      | 27   | 27   | 0      | 1      | 100.00% | <b>100.00%</b> |
| Transitions                 | 60   | 34   | 26     | 1      | 56.66%  | <b>56.66%</b>  |

### Covergroup instance:

**/vi3c\_target\_pkg::i3c\_target\_coverage::daa\_covergroup**

| Summary     | Total Bins | Hits | Hit %   |
|-------------|------------|------|---------|
| Coverpoints | 6          | 6    | 100.00% |
| Crosses     | 1          | 1    | 100.00% |

Search:

| CoverPoints                         | Total Bins | Hits | Misses | Hit %   | Goal %  | Coverage % |
|-------------------------------------|------------|------|--------|---------|---------|------------|
| <a href="#">DAA_ACK_CP</a>          | 1          | 1    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">DAA_BCR_ROLE_CP</a>     | 1          | 1    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">DAA_DCR_CP</a>          | 1          | 1    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">DAA_DYNADDR_CP</a>      | 1          | 1    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">DAA_PID_TOP_BYTE_CP</a> | 2          | 2    | 0      | 100.00% | 100.00% | 100.00%    |

Search:

| Crosses                           | Total Bins | Hits | Misses | Hit %   | Goal %  | Coverage % |
|-----------------------------------|------------|------|--------|---------|---------|------------|
| <a href="#">DAA_DYNADDR_X_ACK</a> | 1          | 1    | 0      | 100.00% | 100.00% | 100.00%    |

### Covergroup instance:

**\i3c\_target\_pkg::i3c\_target\_coverage::target\_covergroup**

| Summary     | Total Bins | Hits | Hit %   |
|-------------|------------|------|---------|
| Coverpoints | 21         | 21   | 100.00% |
| Crosses     | 6          | 6    | 100.00% |

Search:

| CoverPoints                                | Total Bins | Hits | Misses | Hit %   | Goal %  | Coverage % |
|--------------------------------------------|------------|------|--------|---------|---------|------------|
| <a href="#">① OPERATION_CP</a>             | 2          | 2    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">① READDATA_CP</a>              | 6          | 6    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">① READDATA_STATUS_CP</a>       | 2          | 2    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">① TARGET_ADDRESS_CP</a>        | 1          | 1    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">① TARGET_ADDRESS_STATUS_CP</a> | 2          | 2    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">① WRITEDATA_CP</a>             | 6          | 6    | 0      | 100.00% | 100.00% | 100.00%    |
| <a href="#">① WRITEDATA_STATUS_CP</a>      | 2          | 2    | 0      | 100.00% | 100.00% | 100.00%    |

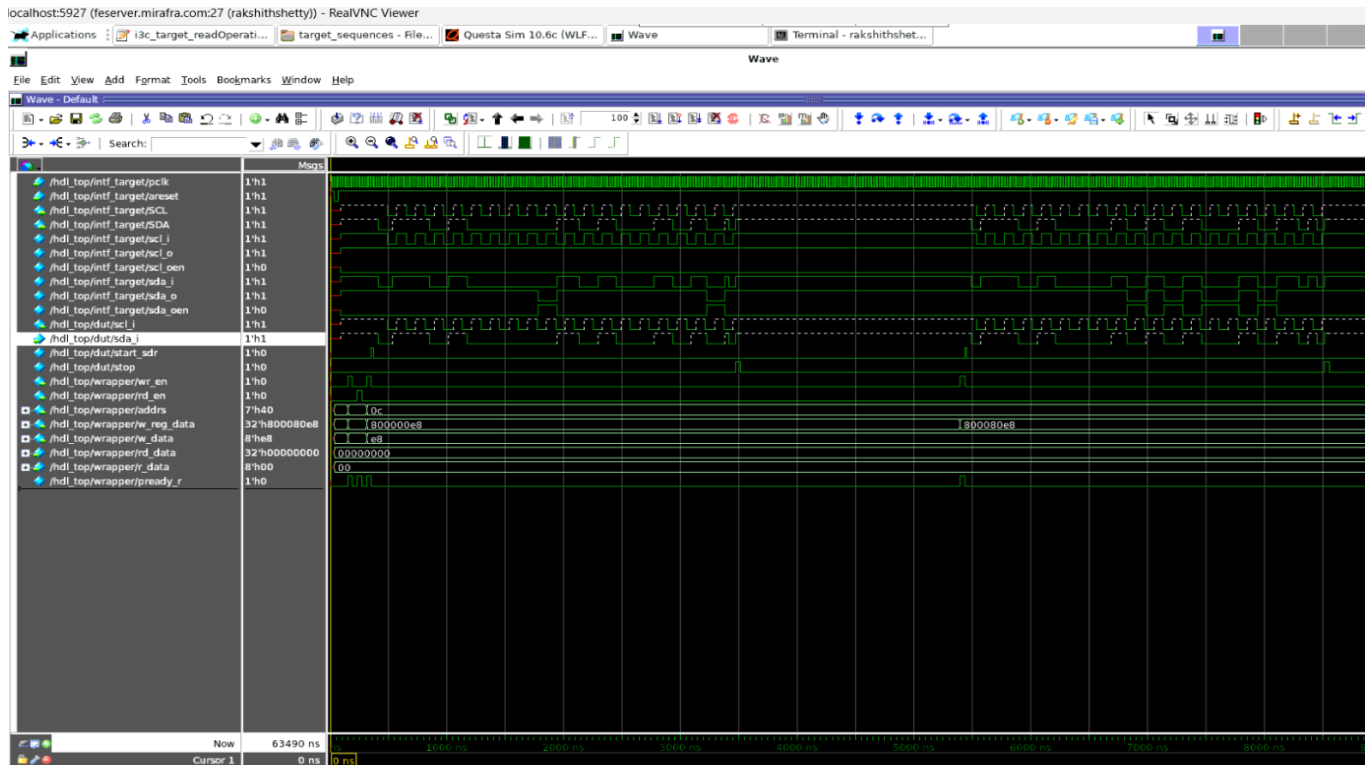
Search:

| Crosses                                       | Total Bins | Hits | Misses | Hit %   | Goal %  | Coverage % |
|-----------------------------------------------|------------|------|--------|---------|---------|------------|
| <a href="#">① OPERATION_CP_X_WRITEDATA_CP</a> | 6          | 6    | 0      | 100.00% | 100.00% | 100.00%    |

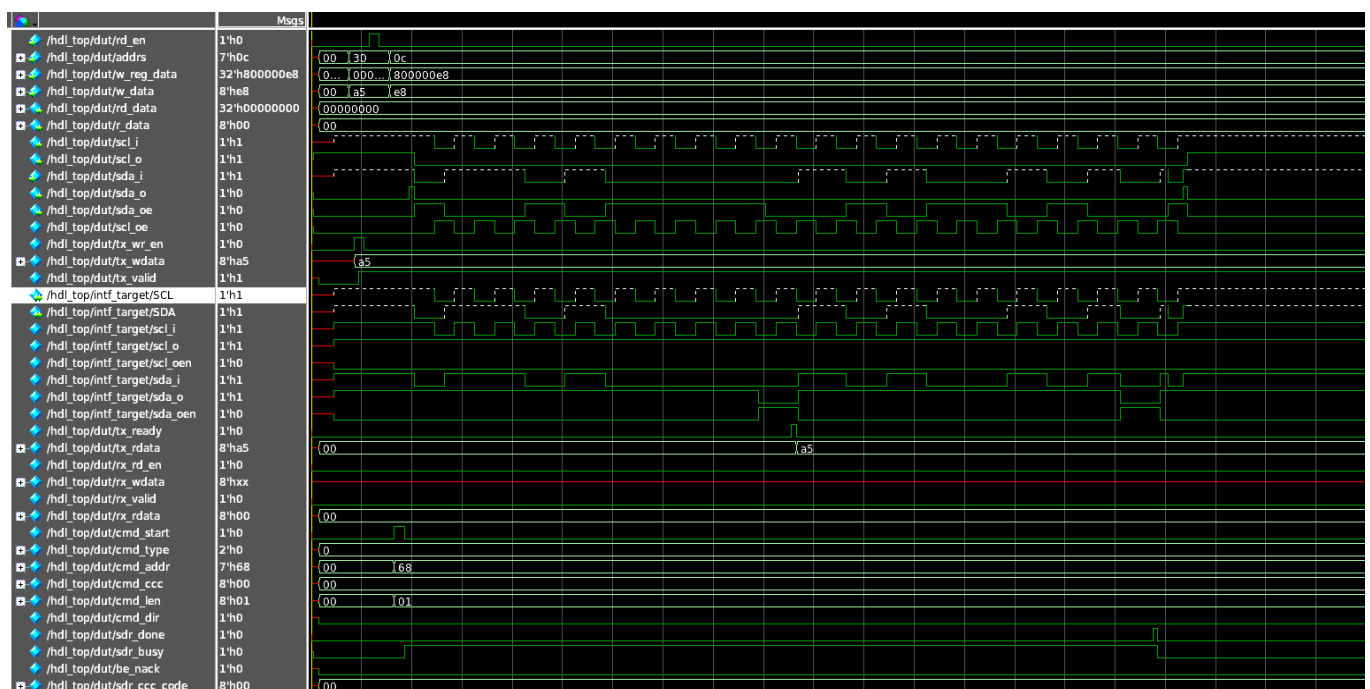
## Chapter – 6

# OUTPUT WAVEFORM

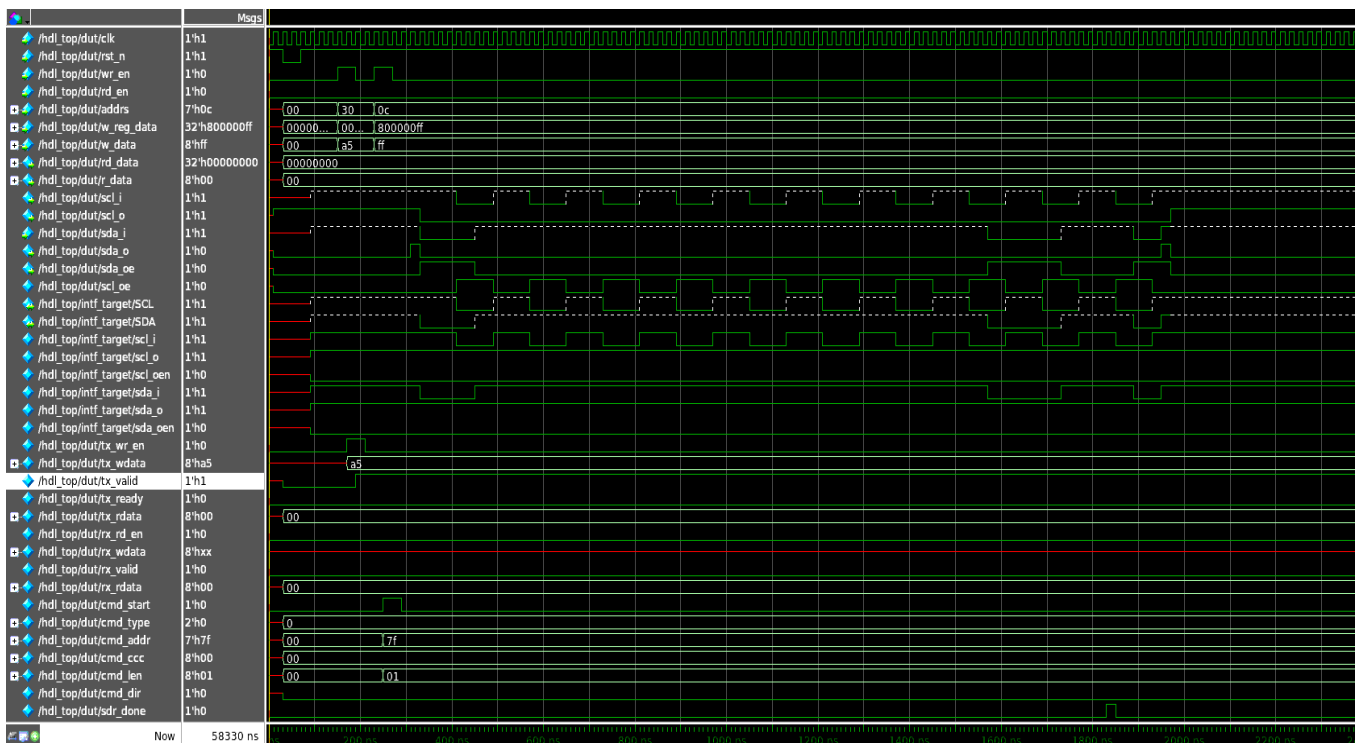
### 6.1 sdr\_write\_read



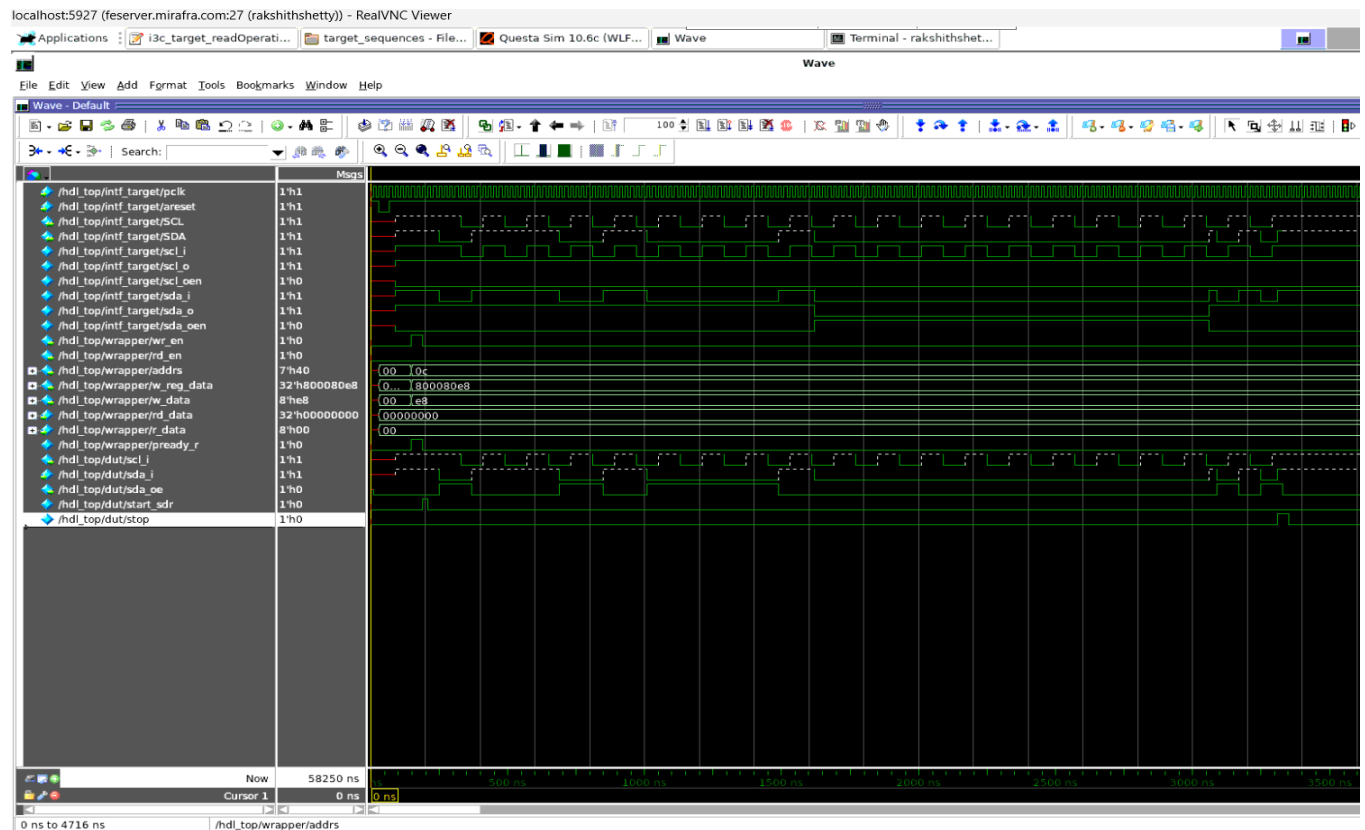
### 6.2 sdr\_write



### 6.3 invalid\_addr\_write

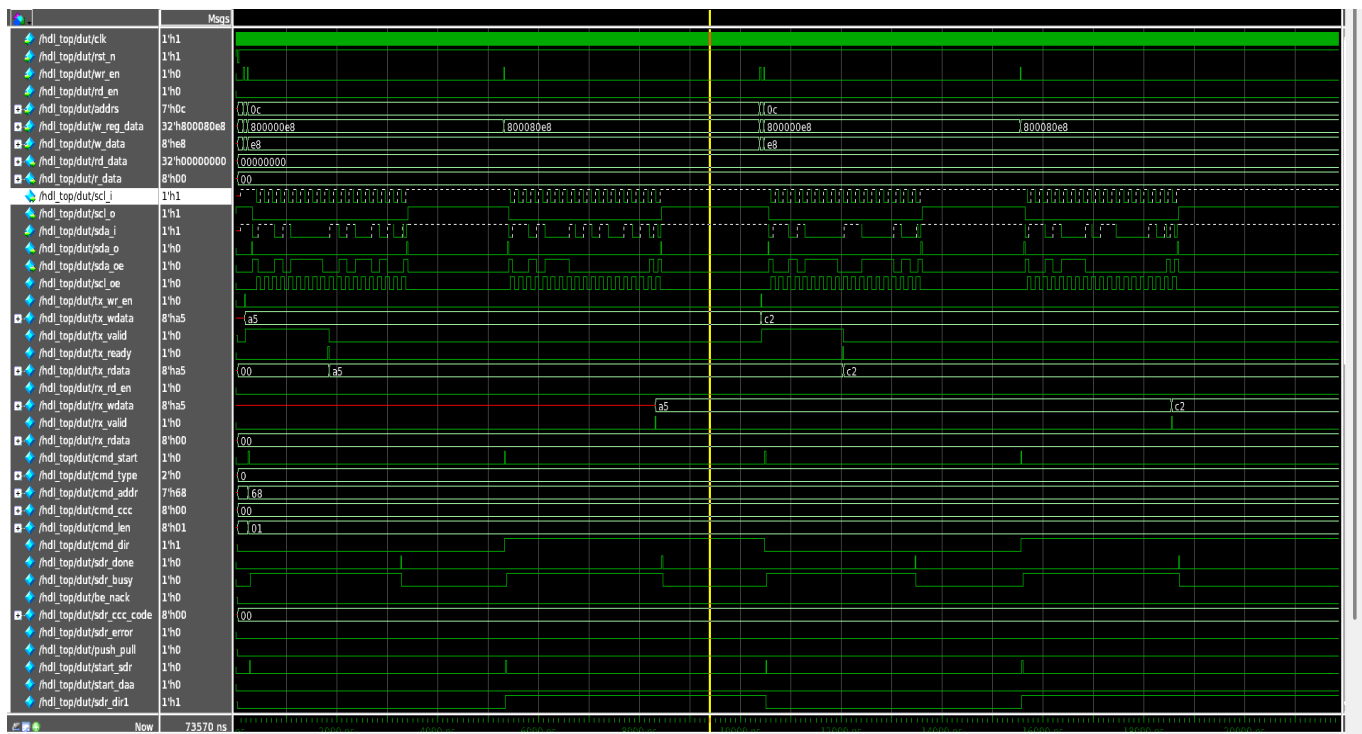


### 6.4 sdr\_read\_test

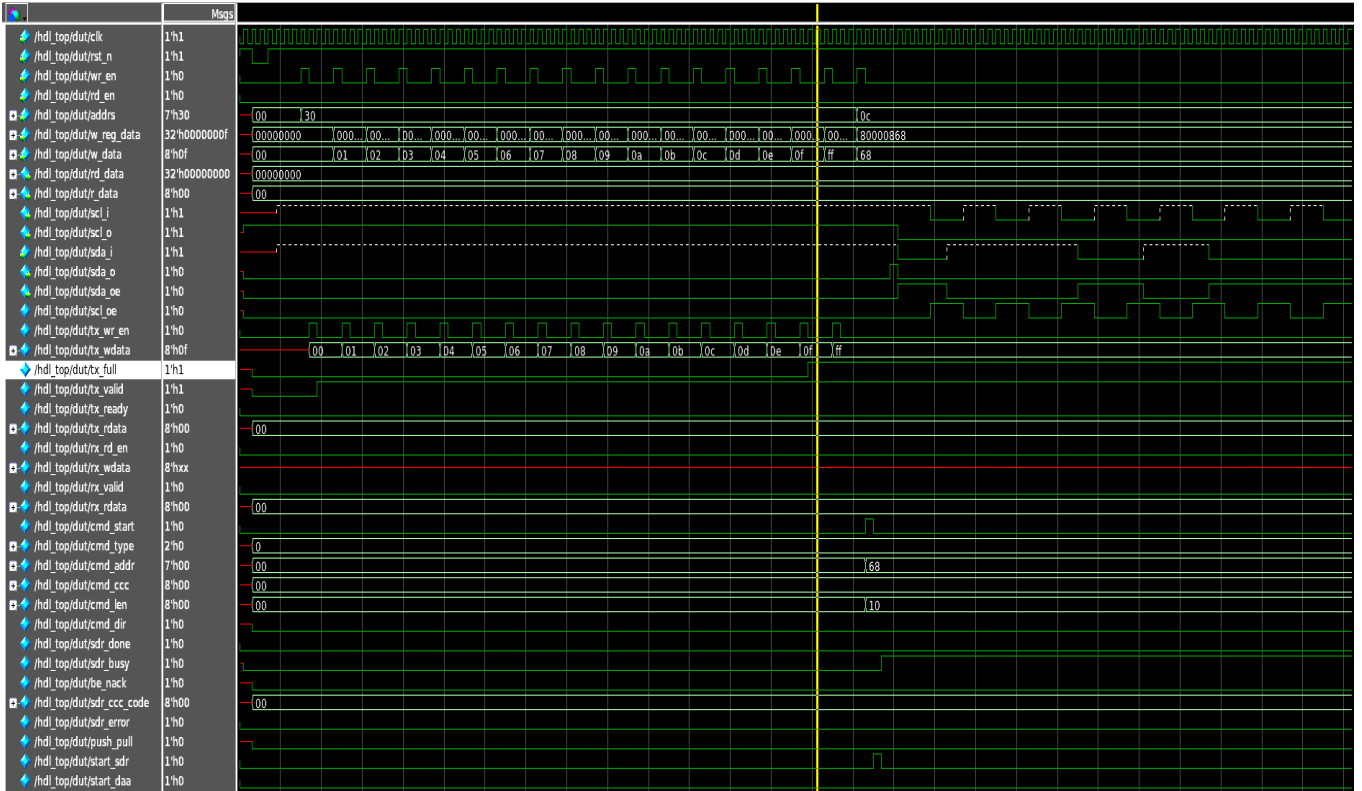




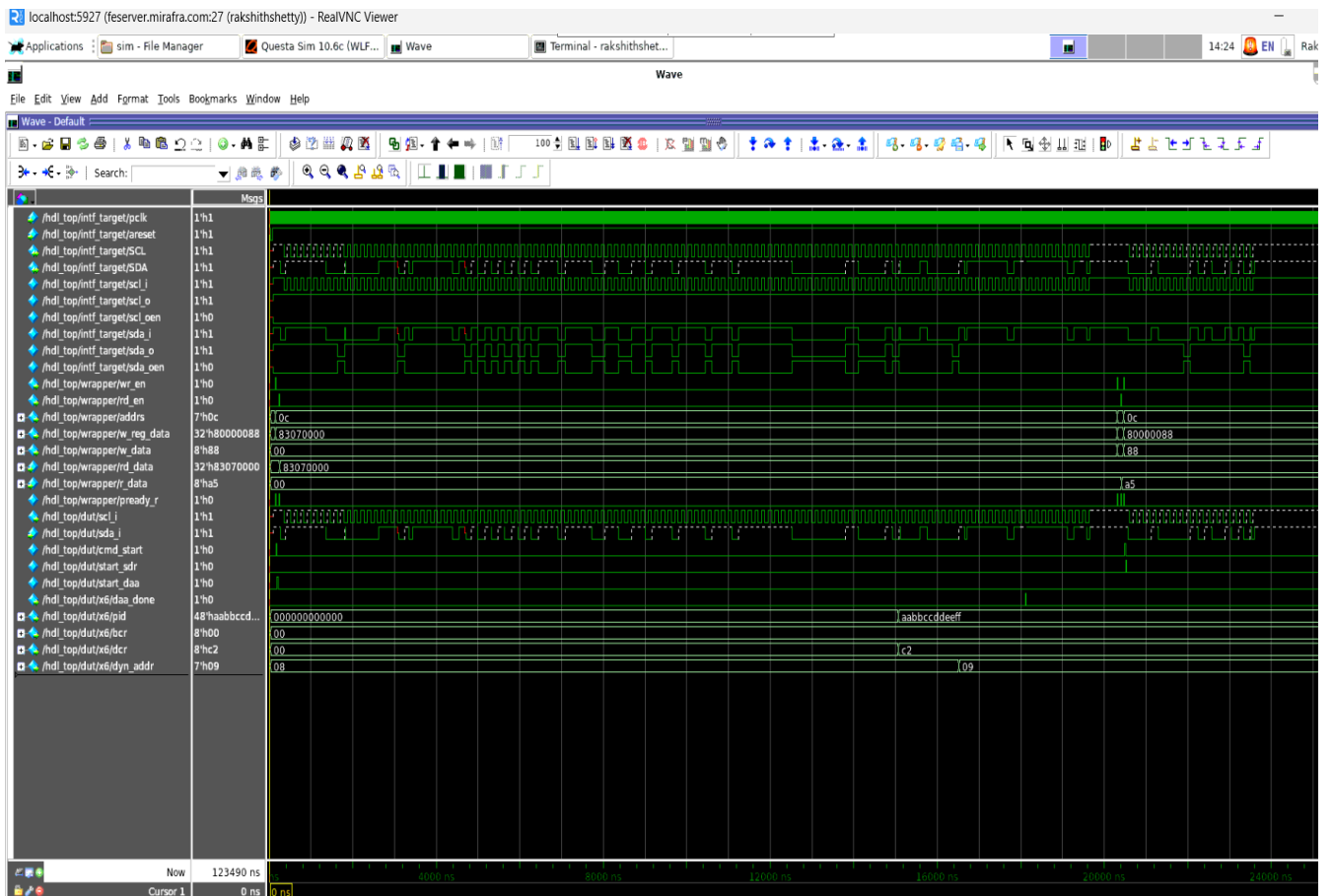
## 6.5 sdr\_write\_read\_write\_read



## 6.6 Tx\_fifo\_full\_check



## 6.7 daa\_write



## CHAPTER – 7

### Conclusion and Learning

- The I3C controller verification environment was developed using the Universal Verification Methodology (UVM), providing a modular and reusable verification structure.
- The verification environment used the I3C AVIP target side and APB AVIP master side for protocol-level verification and DUT register configuration.
- Dynamic Address Assignment (DAA) and SDR read/write operations were verified successfully to validate communication between the controller and target device.
- Driver and monitor BFM s were used to handle low-level I3C and APB signal activity, while the scoreboard verified correctness of transactions and data transfer.
- Functional coverage was implemented to cover different protocol scenarios and improve verification completeness.
- During the verification process, multiple issues such as DAA handling, register readback behavior, and missing register instance declarations were identified and analyzed.
- Through this project, the following key learnings were achieved:
  - o Understanding I3C protocol-level verification including DAA and SDR operations.
  - o Understanding interaction between APB configuration flow and I3C protocol operations.
  - o Improving debugging and analysis skills using simulation logs, waveform analysis, and UVM reporting.
- Overall, this project provided practical exposure to building and debugging a complete UVM-based verification environment for the I3C protocol.

# Links

Git hub link

[Vyas789/i3c\\_avip: Verification IP project for I3C protocol](#)

Verification Link

<https://docs.google.com/spreadsheets/d/1m1WEffA-r7BJRPQbChvKdp94xmzjqiln7RytAiUpFvM/edit?usp=sharing>

Coverage Plan

<https://docs.google.com/spreadsheets/d/1BQ2zNJvKa-Nk6u6NCbHYoCtYxdrqp62X/edit?usp=sharing&ouid=116307063378372675382&rtpof=true&sd=true>